



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №7
Технології розроблення програмного забезпечення
« Патерни проектування.Патерн Спостерігач (Observer)»
«System activity monitor»

Виконав
студент групи ІА–32:
Феклістов Д.С.

Київ 2025

1. Мета роботи

Дослідити призначення та структуру поведінкового патерну проектування **Observer (Спостерігач)**. Реалізувати механізм автоматичного сповіщення компонентів системи (надсилення Email-повідомлень) про критичні зміни стану системи (перегрів процесора) без створення жорстких зв'язків між сенсорами та обробниками подій.

2. Теоретичні відомості

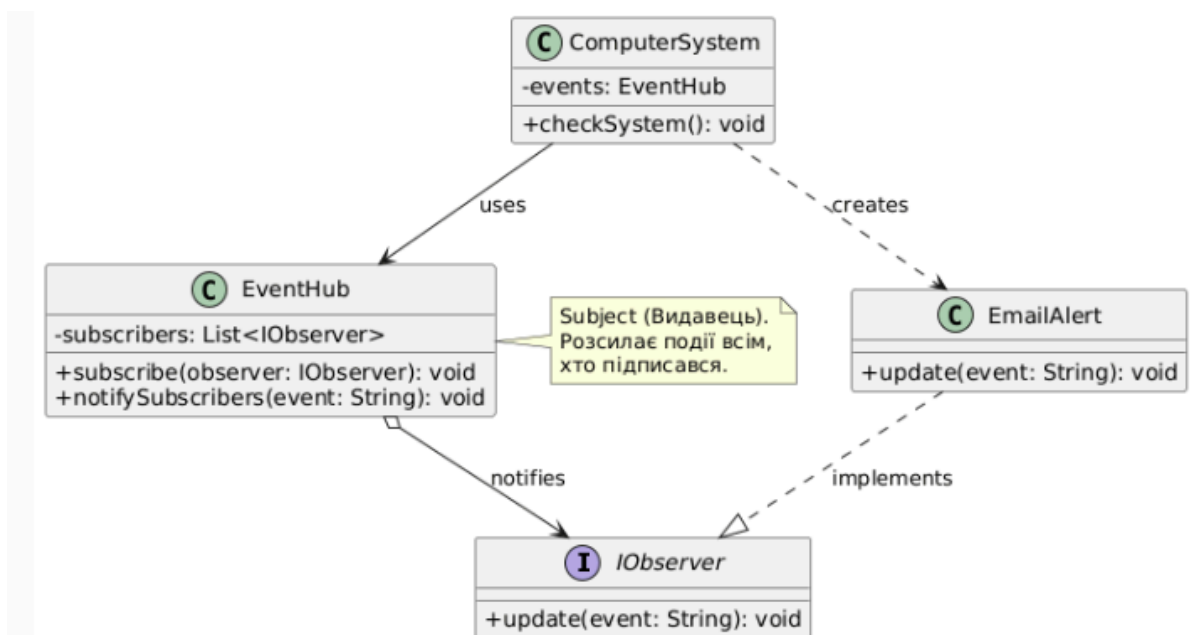
Observer (Спостерігач) — це поведінковий патерн проектування, який визначає залежність типу «один-до-багатьох» між об'єктами таким чином, що при зміні стану одного об'єкта (Subject/Видавець) всі залежні від нього об'єкти (Observers/Підписники) автоматично повідомляються про цю зміну та оновлюються.

Основні учасники:

1. **Subject (Видавець)**: Зберігає список підписників та надає методи для додавання/видалення спостерігачів.
2. **Observer (Підписник)**: Інтерфейс, що оголошує метод повідомлення (наприклад, update).
3. **Concrete Observer (Конкретний підписник)**: Реалізує реакцію на подію.

Переваги: Дозволяє додавати нових підписників (наприклад, SMS-сповіщення, логування в БД) без зміни коду видавця (принцип Open/Closed).

3. Діаграма класів (UML)



4. Хід роботи

1. Створено пакет monitor.observer для реалізації патерна.
2. Розроблено інтерфейс IObservable з єдиним методом update(String event), який мають реалізувати всі підписники.
3. Створено клас EventHub (Subject), який керує списком підписників (subscribe) та має метод notifySubscribers для розсилки повідомлень у циклі.
4. Реалізовано клас EmailAlert (Concrete Observer), який імітує відправку попередження адміністратору при отриманні повідомлення.
5. Інтегровано EventHub у головний фасад системи (ComputerSystem). У метод checkSystem додано логіку: якщо температура CPU перевищує 80°C, автоматично викликається events.notifySubscribers.
6. Проведено тестування: емульовано перегрів системи, в результаті чого в консолі отримано повідомлення про відправку Email.

5. Код програми

Файл: src/monitor/observer/IObservable.java (Інтерфейс)

```
package monitor.observer;

public interface IObservable {
    void update(String event);
}
```

Файл: src/monitor/observer/EventHub.java (Видавець)

```
package monitor.observer;

import java.util.ArrayList;
import java.util.List;

public class EventHub {
    private List<IObservable> subscribers = new ArrayList<>();

    public void subscribe(IObservable observer) {
        subscribers.add(observer);
    }

    public void notifySubscribers(String event) {
        for (IObservable sub : subscribers) {
            sub.update(event);
        }
    }
}
```

Файл: src/monitor/observer/EmailAlert.java (Конкретний спостерігач)

```
package monitor.observer;

public class EmailAlert implements IObserver {
    @Override
    public void update(String event) {
        System.out.println("    >>> [EMAIL] Надсилання адміністратору: " + event);
    }
}
```

Фрагмент інтеграції у src/monitor/facade/ComputerSystem.java

```
package monitor.facade;

import monitor.observer.EmailAlert;
import monitor.observer.EventHub;
import monitor.sensors.CpuSensor;
import monitor.sensors.MemorySensor;
import monitor.utils.SystemLogger;

// ФАСАД: Приховує складність системи
public class ComputerSystem {
    private CpuSensor cpu;
    private MemorySensor memory;
    private SystemLogger logger;
    private EventHub events;

    public ComputerSystem() {
        this.cpu = new CpuSensor();
        this.memory = new MemorySensor();
        this.logger = SystemLogger.getInstance(); // Singleton
        this.events = new EventHub(); // Observer

        // Підписуємо Email-сповіщення
        events.subscribe(new EmailAlert());
    }
}
```

```

    }

    public void checkSystem() {
        logger.log("[ФАСАД] Запуск повної діагностики...");

        // 1. Опитування CPU
        String cpuStatus = cpu.getCpuInfo();
        logger.log(cpuStatus);

        // 2. Опитування RAM
        String ramStatus = memory.getMemoryInfo();
        logger.log(ramStatus);

        // 3. Перевірка температури (Observer test)
        int temp = cpu.getCurrentTemperature();
        if (temp > 80) {
            logger.log("УВАГА! Критична температура!");
            events.notifySubscribers("Перегрів процесора: "
+ temp + "C");
        }

        logger.log("[ФАСАД] Діагностику завершено.");
    }
}

```

6. Результати виконання

```

--- Забір #3 ---
[01:33:24] [ФАСАД] Запуск повної діагностики...
[01:33:24] CPU: Ядер - 12, Завантаження - 8,1%, Температура - 45°C
[01:33:24] RAM (Фізична): 11,1 GB / 13,9 GB | HDD (C:): 56,7 GB / 475,7 GB
[01:33:24] УВАГА! Критична температура!
    >>> [EMAIL] Надсилання адміністратору: Перегрів процесора: 84C
[01:33:24] [ФАСАД] Діагностику завершено.

```

7. Висновок

В ході виконання лабораторної роботи було успішно реалізовано поведінковий патерн **Observer**. Це забезпечило слабку зв'язаність (Low Coupling) між модулем моніторингу температури та модулем сповіщень. Тепер додавання нових реакцій на

критичні події (наприклад, запис в БД або звуковий сигнал) можливе шляхом створення нових класів, що імплементують IObserver, без необхідності змінювати код класу ComputerSystem чи EventHub.

8. Відповіді на контрольні питання

1. **Яке призначення шаблону «Спостерігач» (Observer)?** Визначає залежність типу «один до багатьох» між об'єктами таким чином, що при зміні стану одного об'єкта всі залежні від нього автоматично дізнаються про це та оновлюються.
2. **Яке призначення шаблону «Стратегія» (Strategy)?** Визначає сімейство алгоритмів, інкапсулює кожен з них і робить їх взаємозамінними. Дозволяє змінювати алгоритми незалежно від клієнтів, які їх використовують.
3. **Яке призначення шаблону «Команда» (Command)?** Інкапсулює запит як об'єкт, дозволяючи цим самим параметризувати клієнтів різними запитами, створювати черги запитів або протоколювати їх, а також підтримувати скасування операцій.
4. **Яке призначення шаблону «Ланцюжок обов'язків» (Chain of Responsibility)?** Дозволяє уникнути жорсткої прив'язки відправника запиту до отримувача, дозволяючи декільком об'єктам обробити запит. Об'єкти зв'язуються в ланцюжок, і запит передається по ланцюжку, поки не буде оброблений.
5. **Яке призначення шаблону «Ітератор» (Iterator)?** Надає спосіб послідовного доступу до елементів агрегованого об'єкта (колекції) без розкриття його внутрішнього представлення.
6. **Яке призначення шаблону «Посередник» (Mediator)?** Визначає об'єкт, який інкапсулює спосіб взаємодії множини об'єктів. Сприяє слабкій пов'язаності, утримуючи об'єкти від необхідності посилатися один на одного явно.
7. **Яке призначення шаблону «Знімок» (Memento)?** Без порушення інкапсуляції фіксує і виносить за межі об'єкта його внутрішній стан так, щоб пізніше можна було відновити об'єкт у цьому стані.
8. **Яке призначення шаблону «Стан» (State)?** Дозволяє об'єкту змінювати свою поведінку залежно від внутрішнього стану. Ззовні створюється враження, що змінився клас об'єкта.
9. **Яке призначення шаблону «Шаблонний метод» (Template Method)?** Визначає кістяк алгоритму в операції, залишаючи реалізацію деяких кроків підкласам. Дозволяє підкласам перевизначати певні етапи алгоритму без зміни його структури.

10. **Яке призначення шаблону «Відвідувач» (Visitor)?** Описує операцію, яку необхідно виконати над кожним елементом у структурі об'єктів. Дозволяє визначити нову операцію, не змінюючи класи елементів, над якими вона проводиться.